

CSG-DSL Project Report

Jorge Garcia

April 2026

1 Background

1.1 Constructive Solid Geometry and Signed Distance Functions

Constructive Solid Geometry (CSG) is a modeling system where complex solids are built from simple primitives using boolean operations. The three core operations are union, difference, and intersection.

This project renders CSG models using signed distance functions (SDFs). An SDF returns the signed distance from a point in space to the nearest surface of a shape. Negative values are inside, positive values are outside, and zero is on the surface. SDFs are well-suited to CSG because boolean operations can be implemented with simple min/max combinations.

1.2 Domain-Specific Language Motivation

The project defines a small language, named CSGL, so users can describe geometry at a high level with:

- Primitive constructors such as `Sphere`, `Cube`, `Cylinder`, `Pyramid`, `Cone`, `Torus`, and `Octahedron`.
- Transformations such as `translate`, `rotate`, and `scale`.
- CSG operators `union`, `difference`, and `intersection`.
- A `color(obj, r, g, b)` effect and a `Render(...)` entry point.

The language is compiled into an intermediate representation (IR), then interpreted by a WebGPU shader pipeline. This architecture separates language from rendering.

1.3 Compiler and Runtime Architecture

The implementation follows a staged pipeline:

1. Lexical analysis: convert source text into tokens (including comments and signed numeric literals).
2. Parsing: build an abstract syntax tree (AST) for declarations, expressions, calls, and render statements.
3. Semantic analysis: enforce arity/type rules, definition/use rules, render presence, and emit warnings.

4. IR lowering and optimization: emit a structured CSG IR and optimize transformation/CSG structure.
5. Backend parsing/flattening: parse IR and flatten scene graphs to primitive leaves plus an RPN token stream.
6. WebGPU rendering: evaluate SDF/CSG in WGSL fragment shader through ray marching.

2 What Was Accomplished

2.1 End-to-End Interactive System

This project provides an interactive browser application served by Deno:

- A local server bundles `frontend/app.ts` on demand and serves static assets.
- A Monaco-based editor is integrated with a custom CSGL language definition.
- Compile and Run controls trigger compile-time diagnostics and GPU rendering.
- Runtime interaction includes camera orbit/zoom controls and an optional axis overlay.

2.2 Language Frontend and Semantic Validation

The compiler components implement language behavior:

- Lexer support for comments, separators, operators, and signed floats/integers.
- Parser support for let statements, render statements, nested calls, arithmetic numeric expressions, and a top-level `const {...}` block.
- Semantic checks for:
 - unknown primitives/operations,
 - arity and argument typing,
 - use-before-definition and undefined variables,
 - required render statement,
 - color range validation,
 - warnings for unused variables and values not contributing to final render.

Constant substitution and numeric-expression evaluation are implemented during semantic analysis, which simplifies later lowering and enables expressive scripts.

2.3 IR Design, Lowering, and Optimization

A concrete CSG IR is used as the contract between compiler and renderer. Lowering includes:

- conversion of AST primitives/operations into indexed IR sections,
- matrix generation for translation/rotation/scale,

- propagation of color attributes into IR entries,
- support for multiple render roots.

Optimization includes:

- collapsing transformation chains,
- removing duplicates of equivalent transformed nodes and CSG nodes,
- dead-node elimination by reachability from render roots,
- index remapping to compact final IR.

These steps reduce IR size and avoid redundant work before GPU execution.

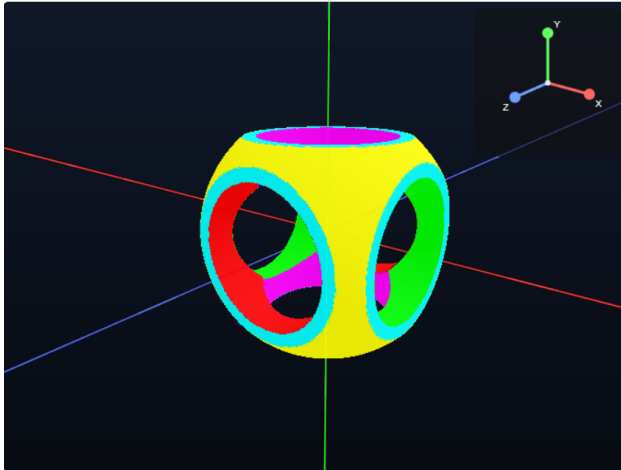
2.4 Backend Parsing, Flattening, and GPU Execution

On the rendering side, the backend accomplishes:

- strict IR parsing with structure and reference validation,
- flattening scene graphs into primitive leaves and a stack-evaluated RPN token program,
- propagation of color attributes down to leaf data,
- conservative step scaling for transformed SDFs,
- scene-radius estimation used for initial camera framing,
- WGSL implementations for the SDFs of the supported primitives,
- shader-side CSG evaluation and normal estimation,
- simple lighting plus per-leaf color blending.

Uniform and storage buffers are structured and uploaded from TypeScript, and runtime controls update camera and axis-overlay state without rebuilding the pipeline.

3 Artifacts



Listing 1: Code for the image above

```
1  const {
2    CYLINDER_R = 0.35;
3    CYLINDER_L = 1.01;
4  }
5
6  let cube = color(Cube(1), 0, 255, 255);
7  let sphere = color(Sphere(0.65), 255, 255, 0);
8  let cs = intersection(cube, sphere);
9
10
11 let c0 = color(Cylinder(CYLINDER_R, CYLINDER_L), 255, 0, 255);
12 let c1 = rotate(
13   color(Cylinder(CYLINDER_R, CYLINDER_L), 0, 255, 0),
14   0, 0, 90
15 );
16 let c0c1 = union(c0, c1);
17
18 let c2 = rotate(
19   color(Cylinder(CYLINDER_R, CYLINDER_L), 255, 0, 0),
20   90, 0, 0
21 );
22 let c = union(c0c1, c2);
23
24
25 let shape = difference(cs, c);
26 Render(shape);
```

References

- [Biscardi()] Chris Biscardi. Intro to signed distance fields. URL <https://www.rustadventure.dev/introduction-to-signed-distance-fields>.
- [for the Web Community Group()] W3C GPU for the Web Community Group. URL <https://webgpu.org/>.
- [Fundamentals()] WebGPU Fundamentals. URL <https://webgpufundamentals.org/>.
- [Heckel(2023)] Maxime Heckel. Painting with math: A gentle study of raymarching, Sep 2023. URL <https://blog.maximeheckel.com/posts/painting-with-math-a-gentle-study-of-raymarching/>.
- [Quilez()] Inigo Quilez. distance functions. URL <https://iquilezles.org/articles/distfunctions/#:~:text=Intro,Euclidean%2C%20L2-norm>.
- [Wong(2016)] Jamie Wong. Ray marching and signed distance functions, Jul 2016. URL <https://jamie-wong.com/2016/07/15/ray-marching-signed-distance-functions/>.
- [Xor(2025)] Xor. Signed distance fields, Feb 2025. URL <https://mini.gmshaders.com/p/sdf>.